

CF_FSNN

*Come funziona la rete
(versione divulgativa)*

Guida pratica all'architettura, al training e ai risultati attuali

Versione	v2 — 2026-05-29
Lettore atteso	Chiunque voglia capire il progetto senza essere esperto di reti neurali spiking
Tempo di lettura	~25 minuti
Versione tecnica	Per dettagli, formule e codice esatto: <i>HOW_IT_WORKS.pdf</i> (v1)

Convenzioni grafiche utilizzate in questo documento:

Box giallo / corsivo	citazione, domanda o riassunto importante
 Box verde	suggerimento pratico
 Box arancione	analogia per capire un concetto
 Box viola	dettaglio tecnico ("per chi vuole approfondire")

1. Cosa fa CF_FSNN, in parole semplici

Immagina un'auto a guida assistita (ACC, *Adaptive Cruise Control*) che segue il veicolo davanti a sé. Il **come** questa auto reagisce — quanto spazio lascia, quanto frena bruscamente, quanto accelera quando il leader accelera — dipende da **5 numeri** che caratterizzano il "carattere" del guidatore (o del software ACC).

Parametro	Significato intuitivo
v_0	A quale velocità mi piace andare se la strada è libera
T	Quanto tempo (in secondi) voglio tenere come distanza di sicurezza
s_0	Anche da fermo, quanto spazio voglio lasciare davanti (in metri)
a	Quanto sono brioso quando accelero
b	Quanto sono morbido quando freno (frenata comoda, non d'emergenza)

CF_FSNN guarda l'auto guidare per qualche secondo e indovina questi 5 numeri.

Più precisamente: la rete riceve in tempo reale 4 informazioni dal sistema V2X (l'auto che parla con le auto vicine) — distanza dal leader, velocità propria, differenza di velocità col leader, velocità del leader — e in uscita stima i 5 parametri del modello fisico ACC-IIDM, lo standard nel settore (Treiber & Kesting 2025).

Perché serve?

Conoscere questi 5 numeri serve a:

- **Capire il tipo di guida** delle auto attorno — utile per la fluidità del traffico in autostrada.
- **Predire le reazioni** di chi ti precede — maggiore sicurezza.
- **Sintonizzare** un controller ACC su uno specifico veicolo o utente.
- **Studiare la stabilità** di un convoglio di auto (string stability).

Analogia. È come ascoltare qualcuno parlare per 30 secondi e capire se è una persona prudente o sportiva. La rete non vede l'auto, vede solo i numeri che descrivono il suo moto — ma da quelli "indovina" lo stile di guida riassunto in 5 parametri.

2. Glossario rapido

Termini che troverai spesso in questo documento, spiegati in modo semplice. Puoi tornare qui ogni volta che incontri una parola sconosciuta.

Termine	Significato in parole semplici
SNN (<i>Spiking Neural Network</i>)	Una rete neurale che, invece di emettere numeri reali, emette impulsi (come i neuroni veri del cervello). Più efficiente da implementare su hardware dedicato.
ALIF neuron	Il tipo di neurone artificiale che usiamo. Si "stanca" se spara troppi impulsi di fila (come i neuroni biologici), e questo aiuta la stabilità della rete.
Surrogate gradient	Un trucco matematico per insegnare alla rete ad imparare anche se la funzione degli impulsi è "spigolosa" (matematicamente non derivabile).
BPTT	Modo standard per allenare reti che hanno memoria nel tempo. Propaga la correzione all'indietro attraverso tutta la sequenza temporale.
Recurrence (low-rank)	La rete riusa il proprio output del passo precedente. "Low-rank" significa che lo fa in modo compatto, risparmiando memoria.
Po2 quantization	I pesi della rete possono assumere solo valori speciali (1, 2, 4, $\frac{1}{2}$, $\frac{1}{4}$ ecc.). Su un chip FPGA, moltiplicare per questi valori = uno shift di bit , gratis.
FPGA	Un chip programmabile, più piccolo ed economico di una GPU. Il nostro target hardware è il PYNQ-Z1 (~300 USD).
V2X	"Vehicle-to-everything": la comunicazione wireless tra l'auto e tutto il resto (altre auto, infrastruttura stradale).
PINN loss	Una funzione di costo che obbliga la rete a essere sia accurata sui dati che coerente con la fisica .
ACC-IIDM	Il modello matematico standard di Adaptive Cruise Control (Treiber & Kesting 2025). Definisce come dovrebbe comportarsi un guidatore razionale.
packet loss V2X	Ogni tanto il messaggio wireless si perde (~2% delle volte). La rete deve essere robusta a questi buchi.

3. Come è fatta la rete

3.1 In una frase

Una piccola rete di 32 neuroni-impulso che ricevono 4 numeri dall'auto, ci pensano per 10 cicli interni, e producono in uscita 5 numeri (i parametri del guidatore).

3.2 A livello di blocchi

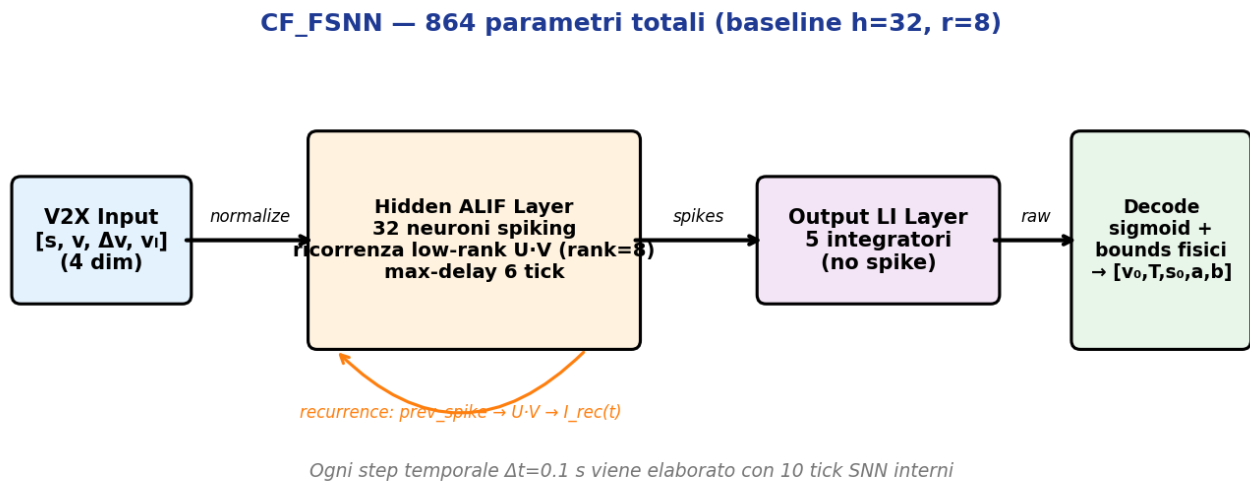


Figura 3.1 — I tre stadi della rete. A sinistra entrano i 4 numeri V2X. Al centro 32 neuroni "spiking" elaborano (con un piccolo ciclo di ri-iniezione del proprio output). A destra 5 numeri grezzi vengono tradotti nei 5 parametri fisici del guidatore.

Ogni "passo" di tempo dell'auto (= 0.1 secondi nella realtà) viene elaborato facendo girare la rete per **10 cicli interni**. Questo dà alla rete tempo di "ragionare" pur restando in tempo reale.

3.3 Quanto è grande?

Cosa	Valore	Confronto
Parametri totali (baseline)	864	Un MLP classico per task simili: 10.000 – 100.000. La nostra è ~ 100× più piccola.
Spazio in memoria	~ 0.7 KB	Sta interamente nella memoria interna (BRAM) del chip FPGA, senza accessi a RAM esterna.
Latenza di inferenza	< 1 ms	Compatibile con controllo ACC real-time.

Perché così piccola? Il nostro target è un chip FPGA economico (PYNQ-Z1, ~300 USD). Se la rete fosse più grande non entrerebbe nella memoria interna del chip, e ogni accesso alla RAM esterna costerebbe in latenza e energia. Piccola = compatibile col vincolo hardware.

4. I tre concetti chiave (con analogie)

4.1 Neurone "che si stanca" (ALIF)

I neuroni veri del cervello hanno un meccanismo di stanchezza: quando sparano molti impulsi di fila, diventano temporaneamente meno reattivi. È un meccanismo di sicurezza che evita attività epilettiche.

Il nostro **neurone ALIF** ha lo stesso comportamento: tiene memoria di quanto ha sparato di recente, e alza temporaneamente la soglia di attivazione. Questo migliora la stabilità del training.

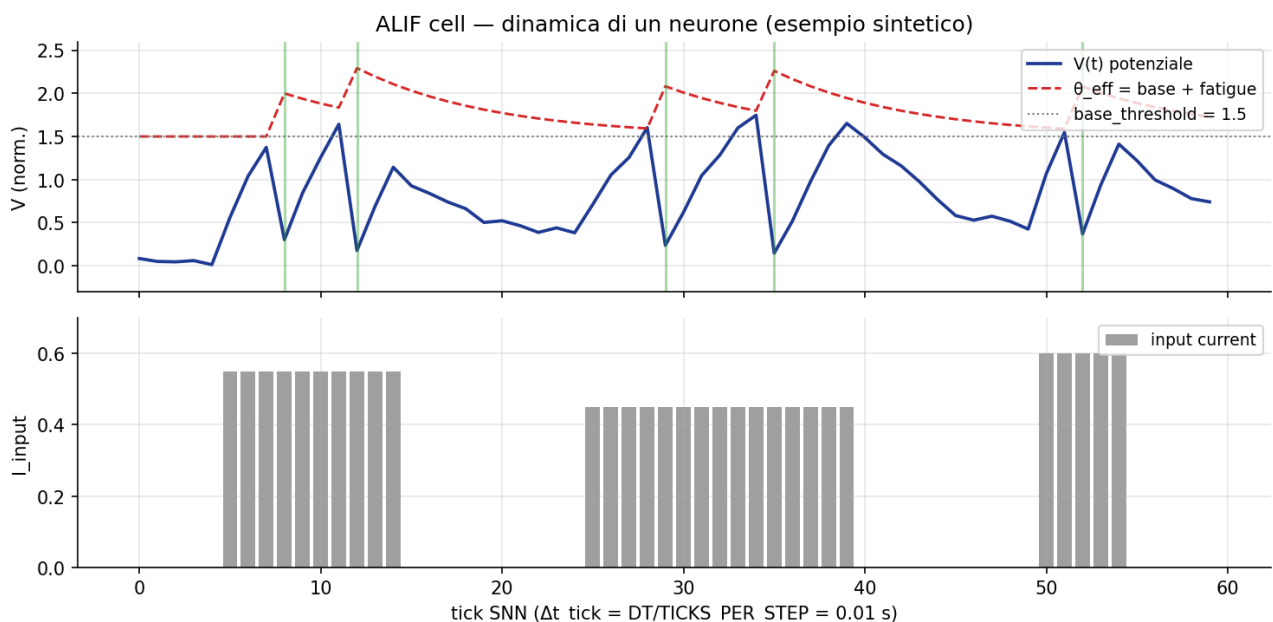


Figura 4.1 — Dinamica di un neurone ALIF. La curva blu è il "potenziale" (carica accumulata); ogni volta che tocca la soglia rossa, parte un impulso (linea verde). Dopo aver sparato, la soglia si alza temporaneamente: il neurone è "stanco" e meno reattivo per un po'.

Per chi vuole approfondire. Tecnicamente è un "Adaptive Leaky Integrate-and-Fire" con soglia adattativa F che decade nel tempo. Le equazioni esatte sono nella versione tecnica v1, §3.1.

4.2 Surrogate gradient (il trucco per imparare)

Il problema: gli impulsi sono **0 oppure 1**, non c'è una "via di mezzo". Matematicamente, la derivata di una funzione "0 oppure 1" è zero quasi ovunque → la rete non potrebbe mai imparare con i metodi classici.

Soluzione: durante l'apprendimento (e *solo* in quella fase), fingiamo che la funzione sia liscia. Sostituiamo la derivata "vera" (che non esiste) con una **curva smussata** centrata sulla soglia. È un trucco di calcolo, ma funziona bene in pratica.

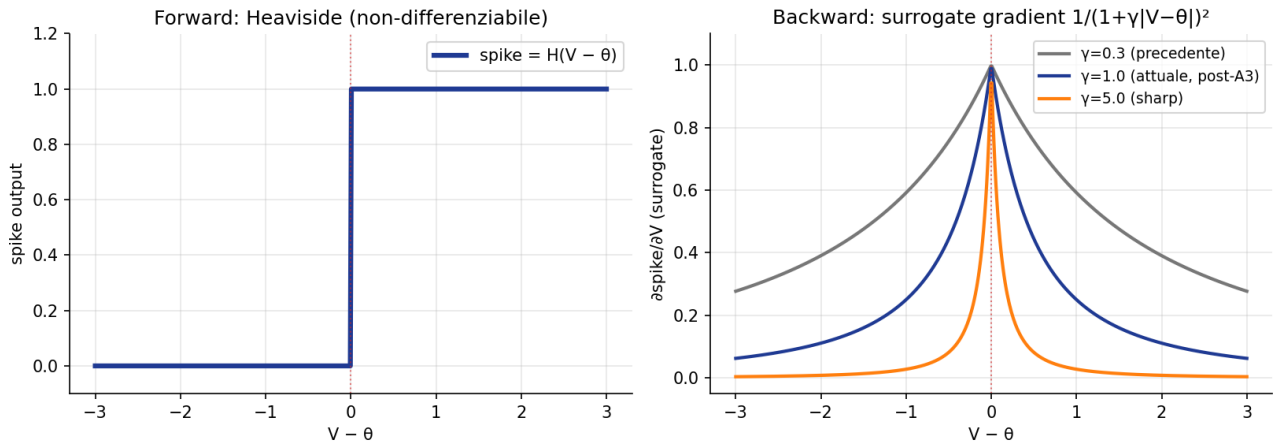


Figura 4.2 — A sinistra: la funzione vera del neurone (0 o 1, gradino). A destra: durante il training fingiamo che la derivata sia una curva liscia. Tre curve mostrano diverse "larghezze" di questa curva (il parametro γ).

L'inferenza poi resta con impulsi 0/1 puri — il trucco serve solo per insegnare.

4.3 Pesì che sono potenze di 2 (per il chip)

Un chip FPGA è molto bravo a fare **shift di bit** (es. $\times 2$, $\times 4$, $\times 0.5$) ma poco bravo a fare moltiplicazioni generiche (è lento, occupa tanta area, consuma tanta energia).

Allora: limitiamo i pesi della rete a essere solo potenze di 2: ± 1 , ± 2 , ± 4 , ± 0.5 , ± 0.25 , ... oppure zero. Risultato: ogni moltiplicazione peso $\times$ input diventa **uno shift di bit**.

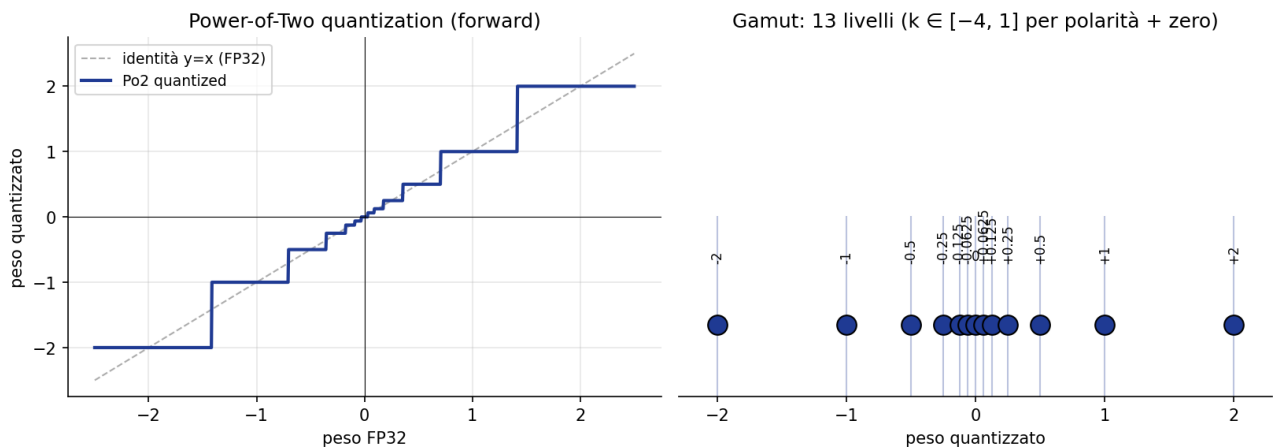


Figura 4.3 — A sinistra: come il "peso reale" viene approssimato dal "peso permesso" (curva blu a gradini). A destra: i 13 valori totali che un peso può assumere (6 positivi, 6 negativi, più lo zero).

Aspetto	Moltiplicazione classica	Con pesi potenze di 2
Area sul chip	~ 100 unità logiche (LUT)	~ 10 unità (10× meno)
Energia per operazione	~ 1 nJ	~ 0.05 nJ (20× meno)
Cicli di clock	4	1
Errore di precisione	zero (reference)	+3 a 8% di errore

Il trade-off: la rete è un po' meno precisa (~3-8% di errore in più) ma sta su un chip che costa qualche centinaio di dollari invece di migliaia. Per noi è un buon affare — la nostra applicazione (controllo ACC) non richiede precisione estrema, richiede economia, bassa latenza e bassa energia.

5. Come la rete impara: la "loss" PINN

Quando si allena una rete, le diciamo *quanto* ha sbagliato tramite un **numero unico** (la "loss"). Più è basso, meglio è. Il modo in cui calcoliamo questo numero determina cosa la rete impara.

La nostra loss combina **5 ingredienti**:

Ingrediente	A cosa serve (intuitivamente)	Peso
L_data	I parametri che predici devono riprodurre correttamente l'accelerazione osservata.	1.0 (massima)
L_phys	Anche quando perdo qualche dato V2X, la fisica deve continuare a tornare.	0.1
L_OU	Il parametro T che predici deve variare nel tempo in modo realistico (non a sbalzi assurdi).	0.05
L_bc	Mai e poi mai predire una distanza minima di sicurezza maggiore della distanza reale (= crash).	1.0
L_sr	I neuroni devono sparare nella misura giusta: né morti, né saturi. Target: 15% di attività.	0.5

I primi due sono "imparare dai dati"; gli ultimi tre sono **vincoli di buon senso fisico** che imponiamo. Il nome "PINN" sta per *Physics-Informed Neural Network* proprio per questo: la rete non impara solo dai dati, ma anche dalle leggi della fisica.

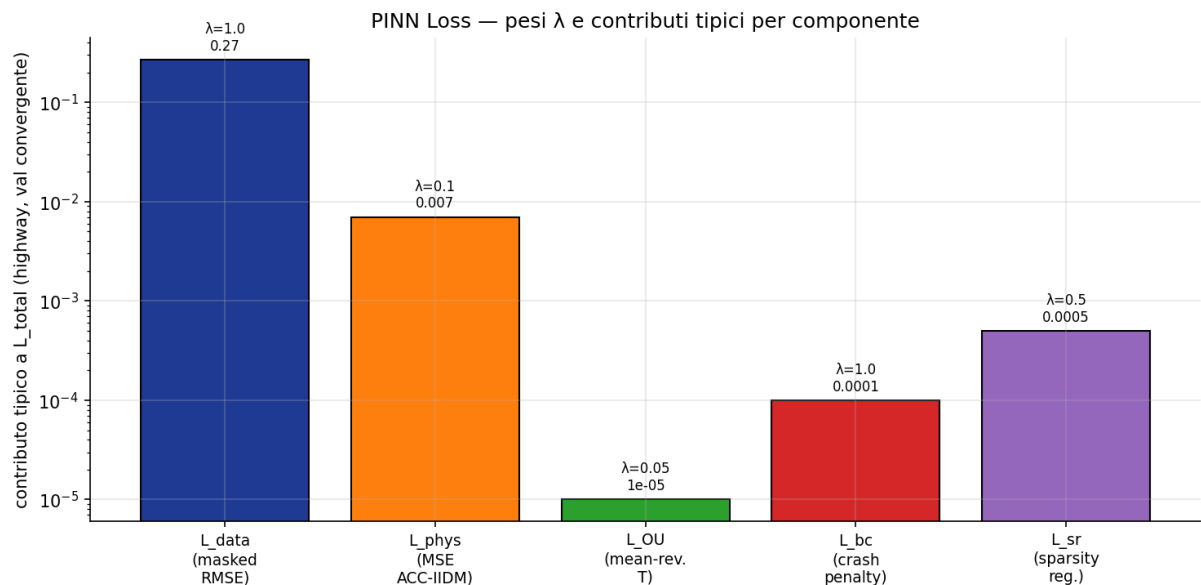


Figura 5.1 — Quanto contribuisce ogni ingrediente alla loss totale in un training "tipico". L_{data} fa il grosso del lavoro; gli altri sono regolatori più piccoli ma importanti.

Perché L_{sr} è importante? Senza questo vincolo, la rete a volte "spegne" tutti i neuroni (li rende silenziosi). Sembra una buona soluzione perché abbassa la loss... ma in realtà rompe completamente il training poco dopo, perché senza impulsi non c'è gradient flow. L_{sr} la tiene "sveglia" alla giusta intensità.

6. Il modello fisico ACC-IIDM (in versione discorsiva)

Tutto il progetto ruota attorno al modello ACC-IIDM di Treiber & Kesting. È un modello che descrive come un guidatore razionale dovrebbe comportarsi in *car-following* (seguendo un veicolo davanti). Funziona così:

6.1 Distanza desiderata

Dato che vado a velocità v , qual è la distanza minima che voglio dal leader?

Il modello risponde:

$distanza_voluta = s_0 + v \cdot T + (\text{piccola correzione se il leader rallenta})$

Cioè: una distanza fissa minima (s_0 , "anche da fermo voglio almeno tot metri") + una distanza proporzionale alla velocità (= "tempo di reazione T moltiplicato per velocità") + una correzione se il leader sta rallentando.

6.2 Accelerazione

Dato che voglio andare a v_0 ma sto a v , e che la distanza voluta è s^ ma sto a distanza s reale, quale accelerazione applico?*

Il modello combina due "spinte":

- una **spinta in avanti** se $v < v_0$ (voglio andare più veloce)
- una **spinta indietro** se $s < s^*$ (sono troppo vicino al leader)

I parametri **a** e **b** decidono quanto bruscamente: a per accelerare, b per frenare.

6.3 Il blend "ACC con CAH"

C'è un dettaglio fine: il modello IIDM standard sarebbe troppo prudente in caso di "cut-in" (un'auto che si infila davanti). Per evitare frenate inutilmente brusche, si **blenda** con un altro modello (CAH, *Constant Acceleration Heuristic*) che assume che il leader continui con la sua attuale accelerazione.

Il blend è governato da un parametro chiamato **coolness** ($c = 0.99$): 99% si fida del CAH, 1% si tiene la sicurezza IIDM. Questo è esattamente ciò che fanno gli ACC commerciali.

Per chi vuole approfondire. Equazioni complete di IDM base, IIDM (versione migliorata che elimina la dispersione a $v \approx v_0$), CAH, e formula esatta del blend con $\tanh$ — vedi versione v1, §8. Riferimento: Treiber & Kesting *Traffic Flow Dynamics* 2nd ed., Ch.12 §12.4.

7. Da dove vengono i dati per allenarla?

Non abbiamo dati reali di auto vere (sarebbe complicato/costoso). Generiamo **dati sintetici** realistici secondo il modello stesso. Per ogni "traiettoria sintetica" (di 100 secondi):

- 1 Scegliamo uno **scenario** casualmente (vedi tabella sotto)
- 2 **Simuliamo l'auto** con il modello ACC-IIDM passo per passo, ogni 0.1 secondi
- 3 Aggiungiamo **rumore realistico**: il 2% delle volte il messaggio V2X si perde (packet loss)
- 4 Ogni tanto (20% delle traiettorie) introduciamo un **cut-in**: un'auto che si infila davanti, gap che si riduce bruscamente

7.1 I 4 scenari

Scenario	Velocità tipica	Carattere	%
highway	~120 km/h	Veloce, time-gap medio, frenata morbida	50%
urban	~54 km/h	Lenta, time-gap stretto, frenata pronta	30%
truck	~80 km/h	Lenta accelerazione, time-gap lungo, range largo	10%
mixed	—	Mix casuale dei 3 sopra	10%

La rete viene allenata su 500-5000 di queste traiettorie. Ogni traiettoria contiene 1000 "fotogrammi" (passi temporali da 0.1 s).

Perché 4 scenari diversi? Per evitare che la rete impari "a memoria" un solo stile di guida. Allenandola su scenari diversi, impara a riconoscere pattern generali del modello ACC-IIDM, qualunque sia la combinazione specifica di parametri.

8. Esempio numerico: un singolo fotogramma

Per fissare le idee, vediamo cosa succede in un singolo passo temporale.

8.1 Situazione

Un'auto sta seguendo un leader in autostrada:

Cosa osservo	Valore
Distanza dal leader (gap)	30 metri
Velocità mia	25 m/s (~ 90 km/h)
Velocità leader	28 m/s (~ 100 km/h)
Differenza Δv	-3 m/s (il leader va più veloce di me)

8.2 Ground truth ("cosa farebbe un guidatore perfetto")

Se questo è un driver in autostrada con parametri standard ($v_0 = 33.3$ m/s, $T = 1.2$ s, $s_0 = 2.5$ m, $a = 1.1$ m/s², $b = 1.5$ m/s²), applicando le formule ACC-IIDM otteniamo:

Accelerazione corretta = +0.74 m/s² (sta accelerando lievemente per recuperare il leader che va più veloce).

8.3 Cosa fa la rete

Se la rete predice i 5 parametri sbagliati — per esempio mette T più alto e a più basso — ricalcolando l'accelerazione con quei parametri otterremmo magari un'accelerazione di -3.5 m/s^2 (cioè una frenata) invece di $+0.7 \text{ m/s}^2$.

L'errore è di circa 4.2 m/s^2 . Questo errore viene incorporato in L_{data} , che propaga la correzione attraverso la rete e aggiusta i pesi.

Ripetendo su milioni di passi temporali (su molte traiettorie, su molte epoche di training), la rete impara a riprodurre il comportamento corretto.

9. Quando possiamo dire che "funziona bene"?

Ci sono due tipi di criteri: numerici e visivi.

9.1 Criteri numerici

Cosa guardare	Funziona bene	Eccellente
Loss totale di validazione	sotto 0.20	sotto 0.15
Errore tipico su v_0	meno di 5.5 m/s (~15%)	meno di 2.2 m/s (~6%)
Errore tipico su T	meno di 0.30 s	meno di 0.10 s
Errore tipico su s_0	meno di 60 cm	meno di 20 cm
Errore tipico su a	meno di 0.33 m/s^2	meno di 0.10
Errore tipico su b	meno di 0.38 m/s^2	meno di 0.13
Spike rate dei neuroni	tra 5% e 30%	tra 10% e 20%
Stabilità del training	nessuna esplosione per 5+ epoche	nessuna esplosione mai

9.2 Criteri visivi (sui grafici diagnostici)

Grafico	Cosa deve mostrare
G5 (scatter T predetto vs T vero)	I punti devono stare vicini alla diagonale (linea verde $y=x$). Se sono dispersi a caso, la rete non sta imparando T.
G7 (violino dei 5 parametri)	Tutte le distribuzioni devono essere dentro le linee rosse/verdi (range fisico). Se escono, la rete sta predicendo valori "non possibili".
G13 (traiettorie ricostruite)	Simulando un'auto con i parametri che la rete predice, la traiettoria deve assomigliare a quella vera per almeno 5 secondi.

9.3 Riferimento di paragone

Non esiste un benchmark "ufficiale" per questo task, ma in letteratura:

- Un **MLP classico** ben calibrato (con metodi tradizionali tipo Levenberg-Marquardt) raggiunge val ~ 0.10 – 0.15 .
- Per la nostra **SNN su FPGA** è ragionevole puntare a val ~ 0.15 – 0.20 (un po' peggio del MLP per la quantizzazione, ma accettabile).
- **Stato attuale: siamo a val ≈ 0.28** — ancora lavoro da fare (vedi §11).

10. Come si lavora con la rete in pratica

10.1 Esperimento singolo

C'è un notebook Jupyter chiamato **Training_File.ipynb**. Si modifica solo la **Cella 1** (la configurazione: nome esperimento, numero di epoche, scenario...), poi si fa "Run All". Il notebook fa tutto da solo:

- 1 Sincronizza il codice col repository
- 2 Esegue un **preflight** — due test mini-veloci (~ 1 min ciascuno): se entrambi passano, ok
- 3 Lancia il training vero
- 4 Mostra i 13 grafici diagnostici
- 5 Mette tutto su Git e fa il push automatico

10.2 Sweep parametrico

Per testare tante configurazioni in batch: **Training_File_Sweep.ipynb**. Stesso schema, ma in Cella 1 si scrive una **lista** di configurazioni da provare in sequenza. Anche qui preflight + train + push, per ogni configurazione.

10.3 Telemetria

Ogni training produce:

File generato	Cosa contiene
training_log.csv	Una riga per epoca, 16 colonne con metriche principali
training_batch_log.csv	Una riga per ogni batch, 20 colonne (~190 KB per epoca). Permette analisi forense anche se il training crasha.
plots/ (13 PNG)	Grafici diagnostici G1-G13 generati automaticamente

Il "per batch" è importantissimo: anche se il training crasha a metà, abbiamo un audit completo di cosa è successo fino al momento del crash. Senza, perderemmo informazioni preziose per diagnosticare il problema.

11. Cosa abbiamo scoperto finora (stato attuale)

11.1 Sweep dimensioni rete

Durante lo sweep **STEP 2B** abbiamo testato 5 reti di dimensioni diverse (da 864 a 9.605 parametri) sullo stesso scenario (highway). Risultato:

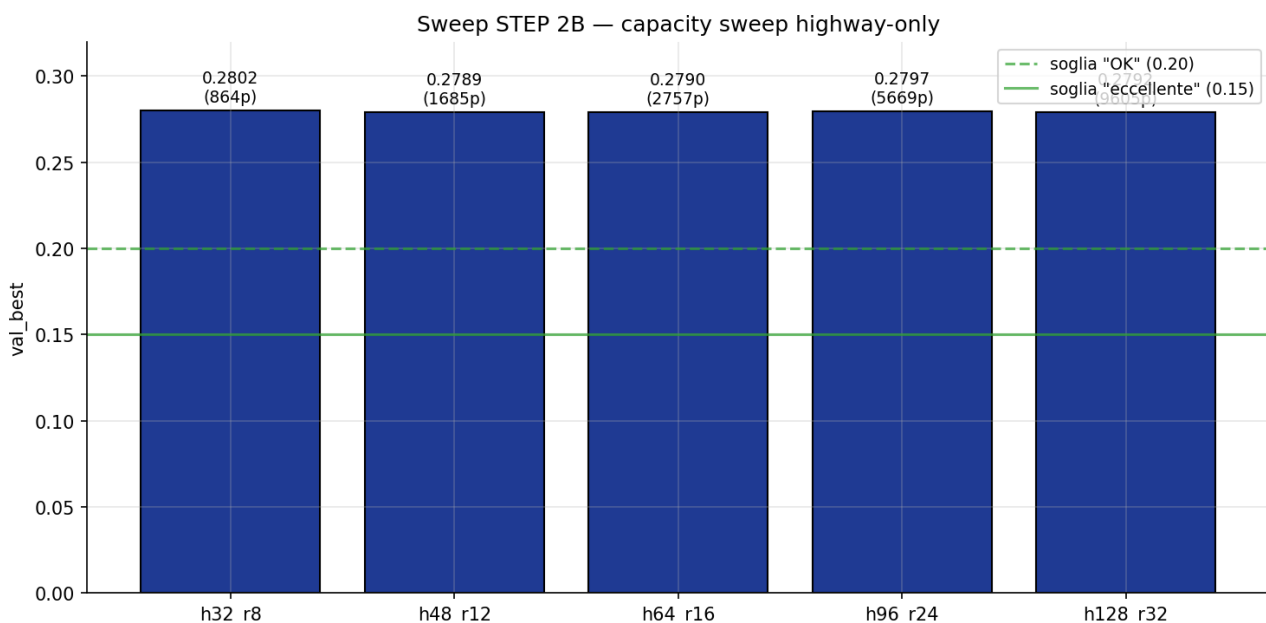


Figura 11.1 — Loss finale di validazione per le 5 reti di diverse dimensioni. Tutte convergono allo stesso valore (~0.279). Le linee verdi indicano le soglie di "funziona bene" (0.20) e "eccellente" (0.15) — siamo ancora lontani.

Tutte le reti, indipendentemente dalle dimensioni, convergono allo stesso valore (~0.279). Questo vuol dire che **ingrandire la rete non aiuta più**: il limite è altrove.

11.2 Cosa stiamo investigando ora

Ipotesi	Cosa testerebbe
Minimi locali	Forse il training si ferma troppo presto. Da testare con più epoche e scheduler "warm restart" (cosine).
Saturazione dati	Forse 500 traiettorie sono troppo poche. Da testare con 1500.
Quantizzazione	Forse i pesi "potenze di 2" stanno mettendo un tetto strutturale di errore.

11.3 Su altri scenari

Abbiamo provato anche urban e truck (sempre con la rete da 2.757 parametri):

Scenario	Loss best	Note
highway	0.279	OK, ma plateau
urban	0.388	Crash a epoca 3 (neuroni morti)
truck	0.160	Crash a epoca 5 (dopo aver imparato bene)

Curiosità: truck è più facile di highway. I parametri di un camion sono più "ristretti" (meno variabilità) → la rete ha meno da imparare. Urban invece è il più difficile perché ha stop-and-go imprevedibili.

12. Per saperne di più

Questo documento è la versione divulgativa. Se vuoi i dettagli tecnici, le formule esatte e il codice, ecco dove guardare:

Vuoi sapere...	Vai a...
Tutti i dettagli tecnici, formule e codice esatto	<i>HOW_IT_WORKS.md / .pdf</i> (versione v1)
Storia delle decisioni passate (perché abbiamo scelto X invece di Y)	<i>TIMELINE.md, P_S.md</i>
Cosa significano i codici "P9", "B5", "F12" ecc.	<i>GLOSSARY.md</i>
Come si esegue un training su Azure	<i>WORKFLOW.md</i>
One-pager per riprendere il lavoro da zero	<i>SESSION_RESUME.md</i>
Il libro di traffic flow di riferimento	Treiber & Kesting, <i>Traffic Flow Dynamics</i> , 2nd ed. (2025)
Il paper sui surrogate gradient	Neftci et al. 2019, <i>Surrogate Gradient Learning in SNNs</i>
Il paper sulla low-rank recurrence	Bellec et al. 2018, <i>Long short-term memory and learning-to-learn in networks of spiking neurons (LSNN)</i>

Spero che questo documento ti abbia aiutato a capire cosa fa CF_FSNN e perché lo fa così. Buona lettura!